

FRD v3.0 — ReporteCobranzas (Antay)

Proyecto: ReporteCobranzas — Cobranzas Antay **Product Owner:** Camilo Ortega F.R.
Versión FRD: 3.0 **Fecha:** 2026-03-16 **Versión app:** v1.8.3 **Estado Supabase:** Migración completa (MIG-000 a MIG-009) **Estado CRM WA:** TIER 1  + TIER 2  completados

Tabla de Contenidos

- 1. Glosario mínimo
- 2. Problema a resolver
- 3. Alcance
- 4. Reglas NO negociables
- 5. Modelo de datos
- 6. Flujo del usuario
- 7. Reglas de conteo
- 8. UI/UX
- 9. Persistencia con Supabase
- 10. CRM WhatsApp TIER 1 (v1.6.0)
- 11. Hotfixes post-TIER 1 (v1.7.x)
- 12. TIER 2 completado (v1.8.x)
- 13. Módulo Clientes Premium
- 14. Quality Gates
- 15. Criterios de aceptación globales
- 16. Control de cambios — Changelog
- 17. **BACKLOG PENDIENTE con diseños** (sección nueva)
- 18. **Matriz de priorización** (sección nueva)
- 19. Ambientes

0. Glosario mínimo

Término	Definición
SSOT	Single Source of Truth — dataset maestro en memoria (<code>df_final</code>)

Término	Definición
Vista filtrada	Dataset para pantalla según filtros activos (<code>df_filtered</code>)
Fresh Load / Ciclo nuevo	Carga nueva de los 2 Excel que reinicia el proceso
Ciclo	Unidad de procesamiento identificada con <code>cycle_id</code> (CIC-YYYYMMDD-HHMM)
Gate	Quality Gate — validación obligatoria antes de declarar un cambio "done"
E2E	End-to-End — prueba completa del flujo real del usuario
Rollback	Reversión al tag estable anterior
Cloud-only	Sin fallback local; si Supabase no responde → bloqueo controlado
Aging	Días de mora de un documento (fecha vencimiento vs hoy)
Lote	Grupo de clientes enviados en un mismo envío WA masivo
Gestión	Registro de contacto/resultado en tabla <code>gestiones</code> (tipo: WHATSAPP, EMAIL, LLAMADA)

1. Problema a resolver

Notificar a clientes por Email y/o WhatsApp usando un Reporte General construido desde 2 Excel externos: 1. Cuentas por Cobrar 2. Cobranzas

La cartera de clientes se mantiene en Supabase (no se recarga por Excel en cada ciclo).

La app debe: - Generar el Reporte General (cliente + documentos por pagar) - Permitir enviar notificaciones por Email y WhatsApp - Mantener trazabilidad de envíos (quién, cuándo, con qué resultado) - Gestionar acuerdos de pago y seguimiento CRM - Permitir reiniciar el ciclo o reiniciar solo el tracking

2. Alcance

2.1 Incluye (IN)

- Carga de 2 Excel y generación de Reporte General
- Cartera maestra mantenida en Supabase (`clientes`)
- Tab Notificaciones Email: selección, preview HTML, envío, reporte post-envío

- Tab WhatsApp: envío masivo, 7 plantillas variables, seguimiento post-lote
- CRM Centro de Gestiones: registro de gestiones, acuerdos de pago, bandeja pendientes
- Trazabilidad completa (ciclos, reconciliación, resumen tablas)
- Persistencia de sesión con auto-restore del último ciclo
- Selector de ciclos históricos
- Módulo Clientes Premium (CRUD completo desde app)
- Ambientes PROD/STAGING con detección automática
- Panel WA de prueba en Tab Configuración (smoke sin datos reales)
- Trazabilidad individual de resultado por cliente en envío masivo
- Persistencia en tiempo real (registro en Supabase cliente a cliente durante el envío)

2.2 No incluye (OUT)

- Rediseñar lógica financiera de deuda/detracción
- Cambiar nombres de columnas clave (prohibido — ver sección 3)
- Cambiar motor de PDF/exportación
- Fallback local (SQLite/session_state) — cloud-only desde v1.6
- Envío automático de mensajes sin intervención del gestor

3. Reglas NO negociables (Anti-regresión)

1. **NO romper el flujo funcional existente.**
2. **NO inventar procesos sin actualizar este FRD.**
3. **NO renombrar columnas clave:**
4. `CodCliente`, `Empresa`, `SaldoReal`, `Correo`, `MATCH_KEY`
5. `ESTADO_EMAIL`, `FECHA_ULTIMO_ENVIO`, `ESTADO_WHATSAPP`
6. Mejoras UI/UX solo si el FRD no pide lógica nueva.
7. Todo cambio debe pasar Gates antes de declararse "done".
8. El agente ejecuta autónomamente análisis, implementación, pruebas y documentación. Solo pide confirmación ante ambigüedad real de regla de negocio no definida en el FRD.
9. **Timestamps en Supabase siempre en UTC real** (`datetime.now(timezone.utc)`).
Nunca hora local Perú como UTC.
10. **Gestiones graban la hora real del clic**, no la hora del envío WA original.

4. Modelo de datos

4.1 Entradas (Excel — 2 archivos)

- **Archivo A:** Cuentas por Cobrar
- **Archivo B:** Cobranzas (Detalle)

La Cartera de clientes NO se carga por Excel en cada ciclo. Se mantiene en Supabase.

4.2 SSOT en memoria

- `df_final` — dataset maestro (SSOT)
- `df_filtered` — vista derivada según filtros activos del Reporte General

4.3 Columnas de envío

- `Correo` — valor crudo del Excel (o de Supabase `clientes.email`)
- `EMAIL_FINAL` — email destino normalizado (minúsculas + trim)
- Regla: la app trabaja por `CodCliente` pero envía a `EMAIL_FINAL`
- Un `EMAIL_FINAL` puede corresponder a múltiples clientes (ej: contabilidad compartida)
- **EMAIL_FINAL no se elimina ni cambia de rol**

4.4 Columnas de tracking (obligatorias)

Columna	Valores	Notas
<code>ESTADO_EMAIL</code>	<code>PENDIENTE</code> / <code>ENVIADO</code> / <code>ERROR</code> / <code>SIN_CORREO</code>	Se inicializa en Fresh Load
<code>FECHA_ULTIMO_ENVIO</code>	timestamp	Vacío hasta confirmación de envío
<code>ESTADO_WHATSAPP</code>	<code>PENDIENTE</code> / <code>ENVIADO</code> / <code>ERROR</code> / <code>SIN_TELEFONO</code>	Ídem

`ESTADO_ENVIO_TEXTO` es campo legacy opcional, no se inicializa en Fresh Load.

4.5 Tablas Supabase

Tabla	Tipo	Descripción
<code>clientes</code>	MAESTRA	Lista oficial. Se carga 1 vez desde Excel, luego se edita desde app
<code>documentos</code>	TRANSACCIONAL	Facturas/boletas. Se recarga en cada ciclo de Excel

Tabla	Tipo	Descripción
cobranzas	TRANSACCIONAL	Aplicaciones (detracciones, amortizaciones, pagos)
notificaciones	TRANSACCIONAL CRÍTICA	Registro permanente de cada envío. NUNCA se borra
gestiones	TRANSACCIONAL	Gestiones CRM (registros de contacto, resultado, metadata)
acuerdos_pago	TRANSACCIONAL	Acuerdos de pago registrados
cuotas_acuerdo	TRANSACCIONAL	Cuotas por acuerdo
ciclos_procesamiento	TRANSACCIONAL	Metadata de cada ciclo cargado
resumen_cliente_ciclo	ANALÍTICA	1 fila por cliente por ciclo
resumen_ciclo	ANALÍTICA	1 fila por ciclo con totales
app_config	CONFIGURACIÓN	Plantillas WA, config SMTP, parámetros

Reglas críticas por tabla: - `clientes` : NO se recarga en cada ciclo - `notificaciones` : NUNCA se borra, se acumula historial - `ciclos_procesamiento` : se acumula, cada ciclo es identificable y recuperable - `cycle_id` formato: `CIC-YYYYMMDD-HHMM` - Timestamps: siempre UTC (`datetime.now(timezone.utc).isoformat()`)

5. Flujo del usuario

5.1 Flujo principal (ciclo diario)

1. Cargar 2 Excel → se genera Reporte General → se crea `cycle_id`
2. Tracking inicial: `ESTADO_EMAIL = PENDIENTE`, `FECHA_ULTIMO_ENVIO = vacío`
3. Filtrar Reporte General → vista `df_filtered` se sincroniza
4. Ir a Tab Email → seleccionar cliente(s) → preview HTML → enviar
5. Ir a Tab WhatsApp → seleccionar plantilla → envío masivo → registrar resultado post-lote
6. Ir a CRM → registrar gestiones, acuerdos, bandeja pendientes
7. La sesión persiste (al día siguiente se ve igual)

5.2 Nuevo ciclo (reemplaza todo)

- Botón "Cargar nuevos archivos" → confirmación 2 pasos
- Al confirmar: se elimina Reporte anterior, tracking vuelve a estado inicial

- Ciclo anterior queda en Supabase, recuperable por selector

5.3 Reiniciar solo tracking (Email)

- En Configuración → "Reiniciar Registro de Envíos"
- Acción: `ESTADO_EMAIL = PENDIENTE`, `FECHA_ULTIMO_ENVIO = vacío`

5.4 Auto-restore

- Al abrir la app: `attempt_auto_restore()` carga automáticamente el último ciclo
- Flag `skip_auto_restore` evita que el auto-restore sobrescriba una elección manual

6. Reglas de conteo (críticas)

- "Enviados Hoy" y "Pendientes" se miden por `CodCliente` único, NO por `EMAIL_FINAL`
- Si un cliente tiene fila Envío WA + fila Gestión → solo se cuenta una vez (deduplicar)
- Monto multimoneda: guardar `DeudaS / DeudaD` explícitos en `wa_details` al enviar
- Fallback: recalcular desde `df_filtered` agrupado por moneda + `CodCliente`
- Historial de gestiones: deduplicar por `RowKey` individual (no por `CodCliente+Tipo`)

7. UI/UX

7.1 Reporte General

- Vista Ejecutiva / Vista Completa
- Pantalla completa real (preserva sesión)
- KPIs en parte superior: Total Saldo (S/ + \$), Total Detracción (solo S/), Total Clientes, Pendientes de Notificar

7.2 Tab Notificaciones Email

- KPIs: "Pendientes de Envío" y "Enviados Hoy" (por `CodCliente`)
- Reporte Post-Envío: obligatorio, persiste entre tabs, tabla con cliente/email/docs/resultado + resumen métricas

7.3 Tab WhatsApp

- Selector de plantilla antes del envío
- Sub-tabs de seguimiento por índice entero (`wa_subtab_idx`) — inmune a cambio de emoji en label
- Panel post-envío: monto `S/ X + $ Y` (no el doble)
- Historial scroll con `max-height: 460px`, header sticky, ordenado por saldo descendente
- Link wa.me por teléfono, badge ↻ Reintentar si `SIN_RESPUESTA`

7.4 Sidebar

- Banner STAGING visible cuando `SUPABASE_URL` contiene URL de staging
- Cabecera con pill "Enterprise" + versión actual
- Ciclo activo: ID legible + timestamp + filas
- Botón "Cambiar ciclo" para volver al selector sin recargar archivos

8. CRM WhatsApp — TIER 1 (v1.6.0, completado 2026-03-13)

RC-FEAT-019: Panel Resultado Post-Envío WA

- Panel de seguimiento post-lote en Tab WhatsApp
- Opciones: `EXITOSO` / `PROMETIO_PAGAR` / `SIN_RESPUESTA` / `ESCALAR`
- Llama a `insert_gestion()` con `tipo_gestion=WHATSAPP`
- Persiste en `gestiones.resultado` en Supabase

RC-FEAT-020: Biblioteca 7 Plantillas WhatsApp

- Selector visual antes del envío masivo
- 7 plantillas: primer aviso, recordatorio, aviso firme, acuerdo, pre-legal, felicitación, solicitud datos
- Variables: `{empresa}`, `{monto}`, `{fecha_venc}`, `{gestor}`, `{PROX_VENC}`
- Editables desde Tab Configuración, guardadas en `app_config`

RC-FEAT-021: Módulo Acuerdos de Pago con Cuotas

- Tablas: `acuerdos_pago` + `cuotas_acuerdo` en Supabase
- Formulario: cliente, monto total, nro cuotas, fecha inicio
- Cálculo automático de fechas de vencimiento
- Timeline visual: `PENDIENTE` / `PAGADA` / `VENCIDA`

RC-FEAT-022: Bandeja de Pendientes del Día

- Lista priorizada: `URGENTE` / `ALTO` / `MEDIO`
- Detecta: WA sin respuesta +48h, cuotas venciendo ≤ 3 días, clientes +30 días mora sin gestión

RC-FEAT-023: Trazabilidad Completa

- Tablas: `resumen_cliente_ciclo` + `resumen_ciclo`
- `reconcile_ciclo_recovery()` en `db_manager.py`
- Documentos desaparecidos → estado `RECUPERADO` + fecha + forma_pago + banco

RC-FEAT-024: Tabs CRM Persistentes al Preparar Nuevo Ciclo

- CRM sigue visible durante la transición de ciclo

RC-FEAT-025: Auto-restore Último Ciclo

- `attempt_auto_restore()` en `app.py` al abrir la app
- Flag `skip_auto_restore` evita sobreescritura por elección manual del gestor

9. Hotfixes post-TIER 1 (v1.7.0 → v1.7.3)

ID	Descripción	Fix
RC-BUG-024	<code>CREATE POLICY IF NOT EXISTS</code> incompatible con PostgreSQL	Bloque <code>DO \$\$</code> en <code>sql/11</code> , <code>sql/12</code>
RC-BUG-025	<code>insert_acuerdo_pago</code> error <code>.select()</code> encadenado	supabase-py sync no lo soporta — eliminado
RC-BUG-026	<code>ESTADO_EMAIL</code> / <code>ESTADO_WHATSAPP</code> en blanco al restaurar	<code>fillna('PENDIENTE')</code> en <code>_docs_to_df()</code>
RC-BUG-027	Selectbox plantilla WA se resetea en cada rerun	<code>key="wa_plantilla_seleccionada"</code> fijo
RC-BUG-028	Variable <code>{PROX_VENC}</code> no disponible en plantillas	Agregada a <code>contact_data</code> y preview

ID	Descripción	Fix
RC- BUG- 029	Botón "Cambiar ciclo" sobrescrito por auto-restore	Flag <code>skip_auto_restore</code> en <code>session_state</code>
RC- BUG- 030	Sub-tab Seguimiento reseteaba al tab 1 en cada rerun	Persistir por índice entero <code>wa_subtab_idx</code>
RC- BUG- 031	Monto WA mostraba solo S/ y doble conteo	Guardar <code>DeudaS / DeudaD</code> explícitos; deduplicar <code>CodCliente</code>
RC- UX- 013	Banner STAGING/PROD en sidebar	Detección automática via <code>SUPABASE_URL</code>
RC- OPS- 006	Ambiente staging configurado	Supabase staging + <code>.env.staging</code>

10. TIER 2 completado (v1.8.0 → v1.8.3, 2026-03-16)

10.1 RC-FEAT-026: Panel WA de Prueba en Tab Configuración

- Input teléfono + textarea mensaje + botón "Enviar WA de prueba"
- Llama a `send_whatsapp_messages_direct()` con contacto ficticio
- No requiere ciclo activo ni `df_final`
- Toast verde en éxito, mensaje descriptivo en error

10.2 RC-FEAT-034: 9 mejoras UX Panel Seguimiento Post-Envío

1. Orden por saldo descendente
2. Teléfono como link `wa.me/{número}` con ícono 
3. KPI % efectividad en tiempo real
4. Tipo de gestión como ícono ( Gestión /  Envío WA)
5. Tooltip notas largas truncadas
6. Barra de progreso del ciclo
7. Color semántico del saldo (rojo $\geq S/5000$, naranja $\geq S/1000$)
8. Badge ↺ Reintentar si resultado = `SIN_RESPUESTA`
9. Tooltip descriptivo en "Guardar todos"

10.3 Serie de fixes WA seguimiento (RC-BUG-032 a RC-BUG-056)

ID	Fix
RC-BUG-032	Notas vacías en historial post-rerun (<code>notas</code> faltaba en SELECT Supabase)
RC-BUG-033	Saldo sin prefijo de moneda — usar <code>DeudaS / DeudaD</code> ya formateados
RC-BUG-043	Historial: deduplicar Envío WA vs Gestión; limpiar nota "WA masivo"
RC-BUG-044	KPI "Mensajes enviados" contar clientes únicos, no filas Supabase
RC-BUG-045	Pendientes: reenvío WA posterior a gestión vuelve a pendiente
RC-BUG-046	Deduplicar cliente por lotes históricos en <code>_cids_sesion</code>
RC-BUG-047	Registrar <code>Hora</code> , <code>Tipo</code> , <code>RowKey</code> en <code>wa_details</code> para casos de reenvío
RC-BUG-048	Duplicados en historial por mutación de <code>session_state</code> en rerun
RC-BUG-049	<code>metadata</code> JSON string sin parsear → <code>mensaje_enviado</code> no visible en historial
RC-BUG-050	(diferido) Join mensaje WA a fila gestión — depende de RC-FEAT-036
RC-BUG-051	Fecha consistente en gestiones manual: timestamp ISO del envío original
RC-BUG-052	Historial de envíos: permite múltiples reenvíos en el mismo ciclo
RC-BUG-053	Duplicados al navegar entre sub-tabs WA — deduplicar por (<code>CodCliente</code> , <code>Tipo</code>)
RC-BUG-054	Revertido: gestiones deben grabar hora del clic, no del envío WA
RC-BUG-055	<code>metadata</code> string parseado ANTES de determinar tipo (Gestión vs Envío WA)
RC-BUG-056	<code>insert_gestion()</code> en QA no aceptaba <code>cycle_id</code> (deploy incompleto)

10.4 Trazabilidad individual + Persistencia real-time

`on_client_sent` **callback:** - `send_whatsapp_messages_direct()` recibe parámetro

`on_client_sent(cod, resultado, contact)` - El callback graba en Supabase

inmediatamente después de cada cliente enviado - Si se corta la luz o el internet a mitad del lote → clientes ya procesados quedan guardados

resultados_por_cliente **dict:** - `send_whatsapp_messages_direct()` retorna `{'exitosos':`

`N, 'fallidos': N, ..., 'resultados_por_cliente': {'000003': 'EXITOSO', ...}}` -

Cada cliente tiene su propio `EXITOSO / FALLIDO` — no el resultado agregado del lote

Timestamps correctos: - `datetime.now()` → hora Perú — solo para display y

`session_state` - `datetime.now(timezone.utc)` → hora UTC real — siempre para

Supabase - `created_at` de Supabase se convierte a hora Perú restando 5h al mostrar en UI

11. Módulo Clientes Premium (v1.7.1)

- Tab `Clientes Premium`: única superficie de mantenimiento de cartera maestra
- CRUD completo: agregar, editar, desactivar clientes desde la app
- Migración desde Excel con reporte de errores
- Home operativo estricto: solo acepta `CtasxCobrar` + `Cobranza`
- Si no hay cartera maestra → ciclo bloqueado con instrucción clara hacia Clientes Premium

12. Quality Gates (obligatorio antes de "done")

Gate	Descripción
Gate 0	Compilación: <code>python -m py_compile</code> pasa sin errores
Gate 1	Tests automatizados: <code>pytest</code> para funciones críticas
Gate 2	Preflight: config/QA mode correcto, no enviar a reales en QA
Gate 3	Smoke manual con evidencia: screenshots de CA-1..CA-5 en staging
Gate 4	Documentación: actualizar FRD + changelog + notas de release

13. Criterios de aceptación globales

ID	Descripción
CA-1	Fresh Load → KPIs Email: Enviados=0, Pendientes>0. Tracking inicial PENDIENTE
CA-2	Filtrar Reporte → Tab Email refleja mismo subconjunto
CA-3	Cientes deuda 0 no aparecen para email (salvo detracción pendiente)
CA-4	Emails compartidos: no bloquea selección, KPIs por CodCliente
CA-5	Post-envío: tracking actualizado, KPIs actualizados, Reporte Post-Envío persiste
CA-6	WA masivo: cada cliente graba en Supabase inmediatamente (no al final del lote)
CA-7	WA masivo: resultado en Supabase es individual por cliente (no todos igual)
CA-8	Timestamps Supabase siempre en UTC real (sin desfase de 5 horas)

14. Control de cambios — Changelog

Versión	Fecha	Cambio
v0.1	2025-12-22	Versión inicial — Email + tracking básico
v0.2	2026-01-15	Estabilización ciclo Email + tracking + UX mínima
v0.3	2026-02-15	Integración Supabase cloud-only, ciclos persistentes
v1.6.0	2026-03-13	TIER 1 CRM WhatsApp completo (141/141 tests)
v1.7.1	2026-03-13	Módulo Clientes Premium + Home 2 archivos + hotfixes SQL
v1.7.2	2026-03-14	Mejoras CRM flow + auto-restore + banner STAGING
v1.7.3	2026-03-15	Fix sub-tab seguimiento WA (RC-BUG-030/031)
v2.0 FRD	2026-03-15	Consolidación FRD completo en repo local
v1.8.0	2026-03-16	TIER 2: RC-FEAT-026 panel prueba WA + RC-FEAT-034 9 mejoras UX
v1.8.1	2026-03-16	RC-BUG-043 a RC-BUG-053: serie fixes WA seguimiento

Versión	Fecha	Cambio
v1.8.2	2026-03-16	RC-BUG-054/055/056: timestamps UTC, metadata parsing, cycle_id
v1.8.3	2026-03-16	on_client_sent + resultados_por_cliente — trazabilidad individual
v3.0 FRD	2026-03-16	Actualización FRD: TIER 2 completado + backlog con diseños

15. BACKLOG PENDIENTE — Con diseños para priorización

15.1 RC-FEAT-027 — Selección automática de plantilla WA por Aging

User story:

Como gestor de cobranzas, quiero que la app me sugiera automáticamente qué plantilla WA usar según los días de mora de cada cliente, para no tener que pensar en eso y asegurar el tono correcto.

Diseño de pantalla:

4. Marketing WhatsApp

Enviar Mensajes

Plantilla WA: [● Recordatorio (sugerida por Aging) ▼]

La plantilla se sugiere según el segmento de mora de los clientes seleccionados. Puedes cambiarla.

#	Cliente	Saldo	Aging	Segmento
1	EMPRESA A SAC	S/ 2,333	22 días	● Recordatorio
2	EMPRESA B EIRL	\$ 4,734	8 días	● Primer Aviso
3	EMPRESA C SAC	S/ 138	45 días	● Aviso Firme
4	EMPRESA D SAC	\$ 3,087	70 días	⊖ Pre-Legal

⚠ Hay 1 cliente en Pre-Legal – revisar antes de enviar

[Enviar WA masivo (4 clientes)]

Regla de negocio: | Aging | Segmento | Plantilla sugerida | Color badge | |---|---|---|---| | 0–14 días | Deuda reciente | Primer Aviso | ● Verde | | 15–30 días | Sin respuesta | Recordatorio | ● Amarillo | | 31–60 días | Mora significativa | Aviso Firme | ● Naranja | | 61+ días | Mora crítica | Pre-Legal | ● Rojo |

Criterios de aceptación: - [] Columna "Aging" + "Segmento" visibles en tabla de selección de clientes - [] Plantilla se pre-selecciona según el segmento más frecuente del lote - [] Gestor puede cambiar la plantilla libremente antes de enviar - [] Alerta visible si hay clientes en Pre-Legal en el lote - [] Sin envío automático — siempre requiere acción explícita del gestor

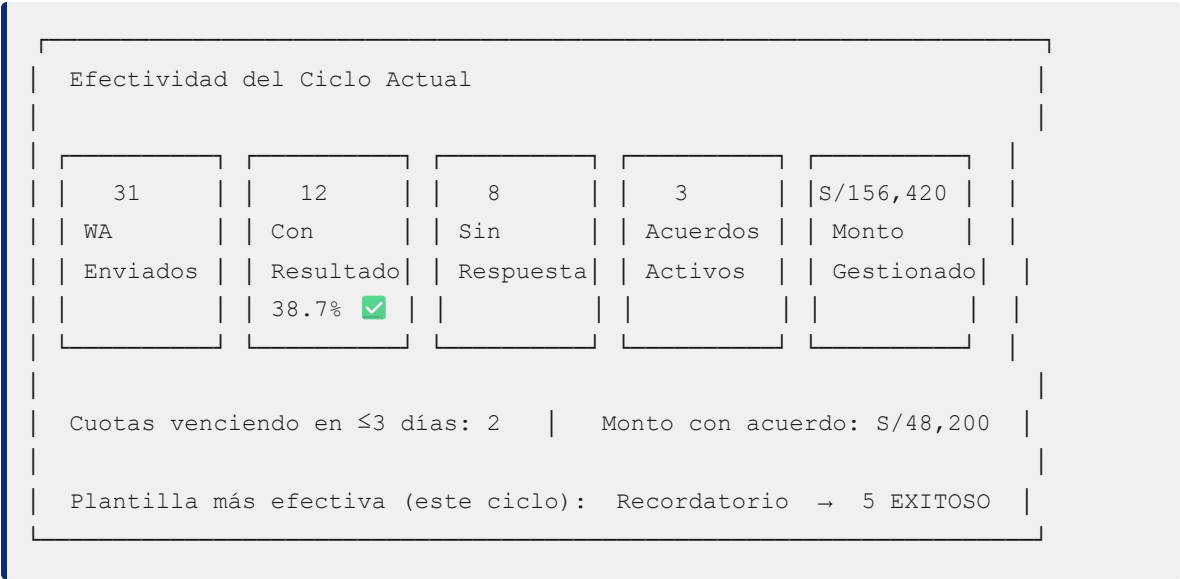
Archivos: `utils/ui/tabs/whatsapp.py` **Esfuerzo:** ~2h | **Prioridad:** P2 | **Tier:** 2

15.2 RC-FEAT-028 — KPIs Expandidos de Efectividad de Cobranza

User story:

Como supervisor de cobranzas, quiero ver en tiempo real qué tan efectiva está siendo la gestión del día (cuántos WA resultaron en compromiso de pago, cuántos acuerdos están activos, cuánto dinero se ha gestionado), para tomar decisiones inmediatas.

Diseño de pantalla:



Criterios de aceptación: - [] KPIs se calculan desde Supabase (`gestiones` , `acuerdos_pago` , `cuotas_acuerdo`) - [] Se muestran en Tab WhatsApp (subtab Seguimiento) y en CRM Centro de Gestiones - [] % efectividad = (EXITOSO + PROMETIO_PAGAR) / total WA enviados - [] No afecta `df_final` ni `df_filtered` (cálculo independiente) - [] Se actualiza al hacer rerun sin recargar ciclo

Archivos: `utils/ui/tabs/whatsapp.py`, `utils/ui/tabs/crm_gestiones.py`, `utils/db_manager.py` **Esfuerzo:** ~3h | **Prioridad:** P2 | **Tier:** 2

15.3 RC-UX-001 — Feedback visual durante envío WA masivo

User story:

Como gestor, quiero ver en tiempo real qué está pasando con cada cliente durante el envío WA masivo (quién ya recibió, quién falló, cuántos faltan), para no quedarme mirando una pantalla en blanco sin saber si el proceso avanza.

Diseño de pantalla:

```

Enviando mensajes WhatsApp...

██████████░░░░░░░░░░░░░░░░ 18 / 31 (58%)

Cliente actual: ALMACENES CHUPACA S.A.C.

✅ GOLOSINAS SEMINARIO E.I.R.L.      S/ 2,333 → Enviado
✅ DISTRIBUIDORA DISUMP S.A.C.      S/ 138 → Enviado
✅ LA GRAN RES SAC                  $ 3,087 → Enviado
⌚ ALMACENES CHUPACA S.A.C.        S/ 18,390 → Enviando...
🟪 EMPRESA E SAC                   S/ 892 → Pendiente
❌ EMPRESA F EIRL                   $ 445 → Error (no tel.)

Enviados: 3 | Fallidos: 1 | Pendientes: 13

```

Criterios de aceptación: - [] Barra de progreso visible con contador `N / Total` - [] Nombre del cliente actual visible mientras se procesa - [] Lista en tiempo real: ☒ Enviado / ☒ Enviando / ☒ Pendiente / ☒ Error - [] Resumen contador al pie: Enviados / Fallidos / Pendientes - [] No bloquea la UI (Streamlit spinner + actualización por callback)

Archivos: `utils/ui/tabs/whatsapp.py`, `utils/whatsapp_sender.py` **Esfuerzo:** ~3h | **Prioridad:** P1 Alto | **Tier:** sin asignar **Nota técnica:** El `progress_callback` ya existe en `send_whatsapp_messages_direct()`. Solo hay que mejorar la UI que lo consume.

15.4 RC-FEAT-001 — Selector tri-modal: Texto / Imagen / Imagen+PDF

User story:

Como gestor, quiero elegir explícitamente qué tipo de mensaje WA envío (solo texto, tarjeta de imagen, o imagen con PDF adjunto), para controlar el formato según el cliente y el dispositivo del destinatario.

Diseño de pantalla:

Modo de envío:

☐ Solo Texto
 ☒ Tarjeta (Imagen)
 ☐ Imagen + PDF

Preview

[Logo Antay]

Estimado EMPRESA A SAC,

Le informamos que tiene una deuda pendiente de S/ 2,333...

[Vista previa de imagen / PDF aquí]

⚠ Modo Imagen+PDF requiere WhatsApp Web abierto con sesión activa

Criterios de aceptación: - [] Radio button tri-modal visible antes del envío - [] UI oculta/muestra preview según selección - [] Lógica de envío respeta estrictamente la selección - [] Advertencia si selecciona PDF y no hay sesión WA activa - [] La selección se guarda en `session_state` (no se resetea en rerun)

Archivos: `utils/ui/tabs/whatsapp.py`, `utils/whatsapp_sender.py` **Esfuerzo:** ~4h |

Prioridad: P1 Alto | **Tier:** sin asignar

15.5 RC-SEC-001 — Seguridad: credenciales en plain text

User story:

Como administrador del sistema, quiero que las credenciales SMTP y de API (Supabase, etc.) NO estén almacenadas en texto plano en archivos JSON accesibles, para proteger la seguridad de la empresa y cumplir estándares mínimos de seguridad.

Problema actual:

`config.json` ← contiene SMTP password, API keys en texto plano
cualquier persona con acceso al servidor puede leerlo

Solución propuesta:

```
.env → variables de entorno (ignorado por .gitignore) ✓
SMTP_PASSWORD=xxxxxx
SUPABASE_KEY=xxxxxx

config.json → solo configuración NO sensible
(nombre empresa, teléfonos, opciones UI)
```

Criterios de aceptación: - [] Ninguna credencial en `config.json` ni en el repo git - [] Variables sensibles cargadas desde `.env` con `python-dotenv` - [] `.env` en `.gitignore` (ya existe) - [] Documentación de variables requeridas en `.env.example` - [] Sin romper config existente en QA (migración con instrucciones)

Archivos: `utils/settings_manager.py`, `utils/email_sender.py`, `.env.example`

Esfuerzo: ~3h | **Prioridad:** P1 Alto | **Tier:** sin asignar **Riesgo:** Requiere ajuste manual en QA para crear `.env` nuevo

15.6 RC-FEAT-036 — Tabla separada `wa_mensajes_enviados`

User story:

Como auditor, quiero poder consultar el texto exacto de cada mensaje WA enviado a cada cliente sin tener que parsear JSON, para generar reportes de auditoría y buscar mensajes específicos rápidamente.

Problema actual:

```
-- Hoy: el mensaje exacto está enterrado en un campo JSONB
SELECT metadata->>'mensaje_enviado' FROM gestiones WHERE ...
-- No se puede indexar, no se puede buscar eficientemente
```

Solución:

```
CREATE TABLE wa_mensajes_enviados (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  gestion_id  UUID REFERENCES gestiones(id),
  cliente_id  TEXT REFERENCES clientes(CodCliente),
  template_label TEXT,
  template_texto TEXT,
  mensaje_exacto TEXT,
  telefono_destino TEXT,
  batch_id    TEXT,
  send_mode   TEXT,
```

```
        created_at      TIMESTAMPTZ DEFAULT NOW()
    );
```

Criterios de aceptación: - [] Tabla creada en Supabase con script SQL versionado - [] `on_client_sent` callback inserta también en `wa_mensajes_enviados` - [] Consulta `SELECT * FROM wa_mensajes_enviados WHERE cliente_id = 'X'` devuelve historial completo - [] Sin cambios visibles en UI (cambio de arquitectura interno)

Archivos: `utils/db_manager.py`, `sql/14_wa_mensajes.sql` **Esfuerzo:** ~5h | **Prioridad:** P1 Alto | **Tier:** 3 **Dependencia:** RC-BUG-050 depende de este ticket

15.7 RC-FEAT-037 — Catálogo editable de resultados de gestión WA

User story:

Como administrador, quiero poder agregar nuevos resultados de gestión (ej: "CLIENTE_INCOBRABLE", "CAMBIO_DE_CONTACTO") desde la UI de Configuración sin necesitar un redeploy de la app.

Problema actual:

```
# Hardcodeado en whatsapp.py:
opciones = ['EXITOSO', 'PROMETIO_PAGAR', 'SIN_RESPUESTA', 'ESCALAR']
# Agregar un nuevo resultado = cambio de código + redeploy
```

Solución:

```
CREATE TABLE catalogo_resultado_gestion (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    codigo_resultado TEXT UNIQUE NOT NULL,
    label_ui          TEXT NOT NULL,
    descripcion       TEXT,
    color_badge       TEXT,
    activo            BOOLEAN DEFAULT TRUE,
    orden             INTEGER,
    created_at        TIMESTAMPTZ DEFAULT NOW()
);
```

Diseño de pantalla (Tab Configuración):

8. Configuración > Resultados de Gestión WA				
Orden	Código	Etiqueta UI	Activo	Acciones

1	EXITOSO	✓	Acordó pagar	✓	[Editar]
2	PROMETIO_PAGAR	👉	Prometió	✓	[Editar]
3	SIN_RESPUESTA	🚫	Sin respuesta	✓	[Editar]
4	ESCALAR	⚠️	Escalar	✓	[Editar]
[+ Agregar resultado]					

Criterios de aceptación: - [] Panel CRUD en Tab Configuración - [] Resultados se cargan dinámicamente desde Supabase en el panel de seguimiento WA - [] Desactivar resultado lo oculta de la UI pero no borra datos históricos - [] Orden configurable (drag or número)

Archivos: `utils/ui/tabs/config_tab.py`, `utils/ui/tabs/whatsapp.py`, `utils/db_manager.py` **Esfuerzo:** ~4h | **Prioridad:** P1 Alto | **Tier:** 3

15.8 RC-FEAT-035 — Link "Ver CRM" desde panel WA

User story:

Como gestor, quiero hacer clic en un cliente del historial WA y que la app me lleve directamente al Tab CRM ya filtrado por ese cliente, para no tener que navegar y buscar manualmente.

Diseño:

#	Cliente	Saldo	Resultado	Acción
1	EMPRESA A SAC	S/ 2,333	✓ Acordó	[Ver CRM →]

Clic en [Ver CRM →] → navega a Tab "Centro de Gestiones" con filtro `cliente_id = '000165'` pre-aplicado.

Criterios de aceptación: - [] Botón/link "Ver CRM" por fila en historial de gestiones WA - [] Clic actualiza `session_state['crm_filtro_cliente']` y cambia tab activo - [] Tab CRM lee el filtro y pre-selecciona el cliente - [] Filtro se limpia al cambiar el contexto manualmente en CRM

Archivos: `utils/ui/tabs/whatsapp.py`, `utils/ui/tabs/crm_gestiones.py` **Esfuerzo:** ~2h | **Prioridad:** P3 Bajo | **Tier:** 3

15.9 RC-BUG-050 — Join mensaje WA a fila gestión en historial

Descripción: En el historial, las filas de tipo "Gestión" no muestran el mensaje WA original que motivó la gestión. Solo se ve en las filas de tipo "Envío WA".

Fix requerido: JOIN `gestiones ← wa_mensajes_enviados` por `cliente_id + batch_id` para recuperar el mensaje exacto.

Dependencia crítica: RC-FEAT-036 debe estar en producción primero.

Esfuerzo: ~2h | **Prioridad:** P2 Medio | **Tier:** 3

16. Matriz de priorización

#	Ticket	Valor negocio	Esfuerzo	Riesgo	Dependencias	Recomendación
1	RC-UX-001 Feedback visual WA	Alto — gestor sabe qué pasa	3h	Bajo	Ninguna	Próximo sprint
2	RC-FEAT-027 Plantilla por Aging	Alto — evita tono incorrecto en Pre-Legal	2h	Bajo	Ninguna	Próximo sprint
3	RC-FEAT-028 KPIs Efectividad	Alto — visibilidad para supervisor	3h	Bajo	Ninguna	Próximo sprint
4	RC-SEC-001 Seguridad credenciales	Alto — riesgo real de exposición	3h	Medio	Ninguna	Urgente (seguridad)
5	RC-FEAT-001 Selector tri-modal	Medio — el modo imagen ya funciona	4h	Medio	Ninguna	Sprint siguiente
6	RC-FEAT-037 Catálogo resultados	Medio — flexibilidad sin redeploy	4h	Bajo	Ninguna	Sprint siguiente
7	RC-FEAT-036 Tabla wa_mensajes	Bajo impacto visible — arquitectura	5h	Bajo	Ninguna	Deuda técnica

#	Ticket	Valor negocio	Esfuerzo	Riesgo	Dependencias	Recomendación
8	RC-BUG-050 Join mensaje WA	Bajo — mejora historial	2h	Bajo	RC-FEAT-036	Después de #7
9	RC-FEAT-035 Link "Ver CRM"	Bajo — conveniencia UX	2h	Bajo	Ninguna	Cuando sobre tiempo

Recomendación de orden de sprints

Sprint actual (semana 1): 1. RC-SEC-001 — seguridad no tiene fecha de vencimiento pero el riesgo es real 2. RC-UX-001 — el gestor lo agradece cada vez que envía 3. RC-FEAT-027 — previene errores de tono con clientes Pre-Legal

Sprint 2 (semana 2): 4. RC-FEAT-028 — KPIs para el supervisor 5. RC-FEAT-001 — selector tri-modal

Sprint 3 (semana 3+): 6. RC-FEAT-037 — catálogo de resultados 7. RC-FEAT-036 + RC-BUG-050 — deuda técnica de arquitectura 8. RC-FEAT-035 — nice-to-have

17. Ambientes

Ambiente	Puerto	Supabase	Arranque
PROD (QA server)	8501	proyecto PROD	<code>5_COMBINADO_APP_Y_TUNEL.bat</code> en servidor QA
STAGING (local)	8502	<code>hrnqngndnohkkegtzgjjg.supabase.co</code>	<code>streamlit run app.py --server.port 8502 con .env.staging</code>

Detección automática: Si `SUPABASE_URL` contiene la URL de staging → banner naranja visible en sidebar.

Deploy manual: 1. Cambios locales → smoke test en STAGING 2. Aprobación del PO 3. Copiar archivos a `\\QA\antay-cobranza\` (deploy) 4. Rerun de la app en QA (automático por Streamlit file watcher) 5. Smoke test en QA 6. Commit + push + merge `dev → main`